

学号：202300130183	姓名：宋浩宇
实验名称：Transformer 机器翻译实验——基于预训练模型的中英文翻译	
1. 实验目的	
1. 理解 Transformer 在机器翻译中的作用。 2. 掌握预训练翻译模型的基本使用方法。 3. 学习使用 HuggingFace Transformers 实现机器翻译。 4. 对比传统规则翻译与神经网络翻译的差异。 5. 分析 Transformer 翻译模型的优点与局限性。	
2. 实验原理	
现代机器翻译广泛采用 Transformer 架构。 Transformer 通过自注意力机制（Self-Attention）学习句子中词与词之间的关系，从而实现更高质量的翻译。 典型翻译流程包括： 1. 输入源语言句子 2. 编码器（Encoder）提取语义表示 3. 解码器（Decoder）生成目标语言句子 例如： 输入：我喜欢机器学习。 输出：I like machine learning. 相比传统规则翻译： Transformer 可以学习上下文语义 能处理复杂句式 翻译更加自然流畅 本实验主要使用现成 Transformer 翻译模型完成中英文翻译任务。	
3. 实验步骤与代码	
实验步骤： 1. 导入 transformers 与 torch 库 2. 加载预训练翻译模型 3. 输入测试句子 4. 对文本进行编码 5. 调用模型生成翻译结果 6. 对输出进行解码 7. 输出英文翻译结果 8. 对多个句子进行测试 9. 分析翻译质量	
代码： <pre>from __future__ import annotations import os from pathlib import Path if "HF_HUB_DISABLE_XET" not in os.environ: os.environ["HF_HUB_DISABLE_XET"] = "1"</pre>	

```

os.environ.setdefault("HF_HUB_DOWNLOAD_TIMEOUT", "600")
BASE_DIR = Path(__file__).resolve().parent
DATA_PATH = BASE_DIR / "exp-8.2-data.txt"
MODEL_NAME = os.environ.get("NLP_WEEK8_MT_MODEL",
                             "Helsinki-NLP/opus-mt-zh-en")
DEFAULT_SENTENCES = [
    "我喜欢自然语言处理。",
    "今天天气很好。",
    "人工智能正在快速发展。",
    "机器翻译非常有趣。",
    "深度学习改变了很多领域。",
]

def read_sentences(path: Path) -> list[str]:
    if not path.is_file():
        return list(DEFAULT_SENTENCES)
    out: list[str] = []
    for raw in path.read_text(encoding="utf-8").splitlines():
        s = raw.strip()
        if s:
            out.append(s)
    return out if out else list(DEFAULT_SENTENCES)

def load_model():
    import torch
    from transformers import AutoModelForSeq2SeqLM, AutoTokenizer
    device = torch.device("cuda" if torch.cuda.is_available() else
                           "cpu")
    tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
    model = AutoModelForSeq2SeqLM.from_pretrained(MODEL_NAME)
    model.to(device)
    model.eval()
    return tokenizer, model, device

def translate_batch(
    tokenizer,
    model,
    device,
    sentences: list[str],
    max_new_tokens: int = 128,
    num_beams: int = 4,
) -> list[str]:
    import torch
    enc = tokenizer(
        sentences,
        return_tensors="pt",
        padding=True,

```

```

        truncation=True,
        max_length=256,
    )
    enc = {k: v.to(device) for k, v in enc.items()}
    with torch.no_grad():
        out_ids = model.generate(
            **enc,
            max_new_tokens=max_new_tokens,
            num_beams=num_beams,
            early_stopping=True,
            do_sample=False,
        )
    texts = tokenizer.batch_decode(out_ids,
skip_special_tokens=True)
    return [t.strip() for t in texts]
def main() -> None:
    print(f"数据文件: {DATA_PATH}")
    print(f"模型: {MODEL_NAME}")
    try:
        tokenizer, model, device = load_model()
    except ImportError as e:
        raise SystemExit(1) from e
    except Exception as e:
        print(f"详情: {e}")
        raise SystemExit(1) from e
    import torch
    sentences = read_sentences(DATA_PATH)
    print(f"\n{'=' * 14} Transformer 中译英 {'=' * 14}")
    # 逐句生成, 便于报告逐条对应;批处理也可, 此处保持输出顺序清晰
    for i, zh in enumerate(sentences, 1):
        en = translate_batch(tokenizer, model, device, [zh],
max_new_tokens=128, num_beams=4)[0]
        print(f"\n--- 句子 {i} ---")
        print(f"中文: {zh}")
        print(f"英文: {en}")
if __name__ == "__main__":
    main()

```

4. 实验结果（图表与输出）

输出结果为：

F:\Homework\自然语言处理作业\venv\Scripts\python.exe "F:\Homework\自然语言处理作业\第八周实验\实验十六-Transformer 机器翻译实验——基于预训练模型的中英文翻译.py"

数据文件: F:\Homework\自然语言处理作业\第八周实验\exp-8.2-data.txt

模型: Helsinki-NLP/opus-mt-zh-en

F:\Homework\自然语言处理作业\venv\lib\site-packages\torch\cuda__init__.py:61:
FutureWarning: The pynvml package is deprecated. Please install nvidia-ml-py instead. If you
did not install pynvml directly, please report this to the maintainers of the package that installed
pynvml for you.

```
import pynvml # type: ignore[import]
```

F:\Homework\自然语言处理作业\venv\lib\site-packages\transformers\models\marian\tokenization_marian.py:176:

UserWarning: Recommended: pip install sacremoses.

```
warnings.warn("Recommended: pip install sacremoses.")
```

Loading weights: 100%[████████████████████] 258/258 [00:00<00:00, 21895.72it/s]

===== Transformer 中译英 =====

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=512) seem to have been set.
`max\_new\_tokens` will take precedence. Please refer to the documentation for more
information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

--- 句子 1 ---

中文: 我喜欢自然语言处理。

英文: I like natural language treatment.

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=512) seem to have been set.
`max\_new\_tokens` will take precedence. Please refer to the documentation for more
information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=512) seem to have been set.
`max\_new\_tokens` will take precedence. Please refer to the documentation for more
information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

--- 句子 2 ---

中文: 今天天气很好。

英文: It's a nice day.

--- 句子 3 ---

中文: 人工智能正在快速发展。

英文: Artificial intelligence is growing rapidly.

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=512) seem to have been set.
`max\_new\_tokens` will take precedence. Please refer to the documentation for more
information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

--- 句子 4 ---

中文: 机器翻译非常有趣。

英文: Machine translation is very interesting.

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=512) seem to have been set.
`max\_new\_tokens` will take precedence. Please refer to the documentation for more
information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

--- 句子 5 ---

中文: 深度学习改变了很多领域。

英文: In-depth learning has changed many areas.

F:\Homework\自然语言处理作业
\.venv\lib\site-packages\huggingface_hub\file_download.py:138: UserWarning:
'huggingface_hub' cache-system uses symlinks by default to efficiently store duplicated files
but your machine does not support them in
C:\Users\23676\cache\huggingface\hub\models--Helsinki-NLP--opus-mt-zh-en. Caching files
will still work but in a degraded version that might require more space on your disk. This
warning can be disabled by setting the 'HF_HUB_DISABLE_SYMLINKS_WARNING'
environment variable. For more details, see
https://huggingface.co/docs/huggingface_hub/how-to-cache#limitations.
To support symlinks on Windows, you either need to activate Developer Mode or to run Python
as an administrator. In order to activate developer mode, see this article:
<https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-device-for-development>
warnings.warn(message)

5. 结果分析

1. Transformer 翻译结果是否符合语义直觉？

相比较于词典法，是的。

2. 哪些句子的翻译效果较好？

第 2、3、4 句相对更好。

3. 哪些复杂句式容易出现翻译错误？

从结果反推，更容易出问题的是带有固定译法或专业术语的表述。例如第 1 句涉及学科名称“自然语言处理”，第 5 句涉及复合专名“深度学习”，模型在术语选择上出错，说明一旦遇到专名、多字复合词或与训练语料分布不一致的搭配，即使句子不算很长，也可能译错。一般意义上，从长句到从句嵌套、多指代、多修饰的句子，解码更容易漏译或错配。

4. 与规则翻译相比，Transformer 翻译有哪些优势？

相比词典法那种逐词查表拼接，本次模型能输出完整英文句子，并处理词典外的词，例如第 2 句的天气、第 3 句的人工智能等，没有出现未登录占位。词序和语法更接近英语习惯，例如第 2 句用常用口语表达概括天气。整体上能利用上下文选词造句，而不是只做片段替换。

5. 模型是否会出现语义遗漏或语义错误？

会出现语义错误。

6. 长句翻译是否比短句更困难？为什么？

通常更长、结构更复杂的句子更难，因为依赖关系更远，注意力与解码要在更长序列上保持指代和修饰一致，一步选错容易连带后面都错。你这次语料以中短句为主，难句体现不充分，

但从第 1、5 句在专名上的失误可以类推，序列稍长、信息更密时，错误风险往往会升高。

7. 为什么 Transformer 更适合机器翻译任务？

机器翻译需要整句建模和灵活的词序与用词选择。Transformer 用自注意力在全局范围内建立词与词之间的关联，便于编码端把握语义，解码端逐步生成目标语，比纯词典规则更能处理未登录词、搭配和句法重组。因此在这类端到端的句子级翻译上，通常比仅靠词典与固定顺序的规则方法更合适。你这次的第 2、3、4 句就是这种优势的体现，同时第 1、5 句也说明它仍受语料与模型容量限制，并非不会错。

6. 实验总结

本次加载 HuggingFace 上的 Helsinki-NLP/opus-mt-zh-en 模型，对五句中文逐句生成英文。程序已跑通并完成推理，第 2、3、4 句整体语义自然，如天气句、人工智能句、机器翻译句的英文与直觉较一致。第 1 句将“自然语言处理”译成 `naturallanguagetreatment`，第 5 句将“深度学习”译成 `In-depthlearning`，存在术语或搭配偏差，说明预训练模型仍会偶发语义错误。控制台另有 `transformers` 生成参数与缓存符号链接等提示，不影响本次得到完整译文结果。